What is a Kubernetes Job?

A **Kubernetes Job** is used to **run a task until completion.**

Unlike Deployments (which run continuously), **Jobs are finite workloads.**

Examples:

- Database migration
- Backup
- Batch processing
- Report generation
- One-time scripts

Why do we need Jobs?

❌ **Problem with Pods**

- Pod runs once
- If it fails → no retry
- No success/failure tracking

✅ **Job solves this**

- Retries on failure
- Tracks completion status
- Ensures task finishes successfully

Job vs Pod vs Deployment

Feature	Pod	Job	Deployment
Retry on failure	❌	✅	✅
Completion tracking	❌	✅	❌
Long-running	❌	❌	✅
Batch workloads	❌	✅	❌

Job Architecture & Internals

Components involved:

- **Job Controller** (kube-controller-manager)
- **Pod(s)** created by Job
- **kubelet** executes Pod
- **API Server** stores Job status

Job Lifecycle (Internals Flow)

---

1. Job object is created
2. Job Controller creates Pod(s)
3. Pod runs the task
4. Pod exits with:
    - exit 0 → success
    - non-zero → failure
5. Job Controller:
    - retries if allowed
    - marks Job Complete or Failed

Basic Job Example (Hello World)

```
apiVersion: batch/v1
kind: Job
metadata:
 name: hello-job
spec:
 template:
  spec:
   containers:
   - name: hello
    image: busybox
    command: ["sh", "-c", "echo Hello Kubernetes && sleep 5"]
   restartPolicy: Never
```

Apply:

```
kubectl apply -f job.yaml
```

Check status:

```
kubectl get jobs
kubectl get pods
kubectl logs <pod-name>
```

restartPolicy (MANDATORY RULE)

Jobs allow **only**:

> restartPolicy: Never
>
> restartPolicy: OnFailure
>
> ❌ Always is NOT allowed

Job Completion Modes

1 Non-Parallel Job (Default)

- One Pod
- Runs once
- Succeeds or fails

```
spec:
  completions: 1
  parallelism: 1
```

2 Parallel Jobs (Fixed Completion Count)

Used when:

- Large dataset
- Chunk processing

```
spec:
  completions: 5
  parallelism: 2
```

Max 2 pods run at a time
Total 5 successful completions required

3 Work Queue Pattern (Advanced)

- Pods pull tasks from queue (SQS, Redis)
- Completion not pre-known

```
spec:
  parallelism: 5
  completions: null
```

Pods exit when queue is empty.

Job Failure Handling

backoffLimit (Retries)

spec:

---

backoffLimit: 3

Means:

- Pod retried max **3 times**
- After that → Job marked Failed

Pod Exit Codes

Exit Code	**Meaning**
0	Success
non-zero	Failure

🔹 **ActiveDeadlineSeconds (Timeout)**

Kill Job if it runs too long:

spec:

activeDeadlineSeconds: 120

✔ Useful for stuck jobs

🔹 **TTL After Job Completion (Cleanup)**

Automatically delete finished jobs:

spec:

ttlSecondsAfterFinished: 300

✔ Avoids clutter in prod clusters

🔹 Indexed Jobs (Advanced & Modern Feature)

Each Pod gets a unique index.

```
spec:
  completionMode: Indexed
  completions: 3
```

Each Pod gets:

JOB_COMPLETION_INDEX=0|1|2

Use case:

- Sharded batch processing
- Data partitioning

🔹 CronJob (Job Scheduler)

CronJob = **Scheduled Job**

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: busybox
            command: ["sh", "-c", "echo backup running"]
          restartPolicy: OnFailure
```

CronJob Internals

- CronJob Controller creates Job
- Job Controller creates Pods
- Old Jobs cleaned based on history limits

CronJob Important Fields

successfulJobsHistoryLimit: 3

failedJobsHistoryLimit: 1

concurrencyPolicy: Forbid | Allow | Replace

🔹 **Job with ConfigMap & Secrets (Real Example)**

```
envFrom:
- configMapRef:
    name: app-config
- secretRef:
    name: db-secret
```

🔹 **Job with Volume (Backup Scenario)**

```
volumes:
- name: backup
  persistentVolumeClaim:
    claimName: backup-pvc
```

🔹 **Debugging Jobs (PRODUCTION)**

Check job status

kubectl describe job <job-name>

See failed pods

kubectl get pods --selector=job-name=<job-name>

Pod logs

kubectl logs <pod-name>

🔹 **Common Real-World Scenarios**

1️⃣ **DB Migration Job**

- Run once during deployment
- Ensures schema consistency

2️⃣ **Backup Job**

- Nightly CronJob
- Uploads to S3/GCS

3️⃣ **Data Processing**

- Parallel Jobs
- Large CSV / ETL tasks

4️⃣ **CI/CD Pre-deployment Validation**

- Smoke tests
- Linting
- Security scans

🔹 **Job vs CronJob vs DaemonSet (Interview Trap)**

Feature	Job	CronJob	DaemonSet
One-time	✅	❌	❌
Scheduled	❌	✅	❌
Per node	❌	❌	✅
Batch	✅	✅	❌

🔹 **Common Mistakes (VERY COMMON)**

❌ Using Deployment for batch jobs
❌ Forgetting TTL cleanup
❌ No backoffLimit → infinite retries
❌ Large parallelism → resource starvation

Kubernetes Jobs are used for finite, batch workloads. The Job controller ensures Pods run to completion with retries, parallelism, timeout control, and cleanup, making Jobs ideal for migrations, backups, and batch processing.